

For bkhack, the user gets to familiarize with the concept of *stream-based programming*. Indeed, visitors of the site will often catch glance of command hints and tips as they are littered about the user interface. We assume that the users are new-comer computer-science interests, we want to teach them about stream-based programming and its aspects.

Stream programming in bkhack is sh pipelining [She], with certain flavorful differences¹. The most important difference is that, instead of dealing with textual data and lines and files, here we deal with abstract post data and post counts. Consider

```
feed | split -c 10
```

where `split` has the option `-c NUM` where `NUM` is the count size of each chunk. Conventionally, the POSIX `split` command accepts the option `-l NUM` where `NUM` is line number size of chunk.

Another difference is that there is no filesystem, no concept of “space”. There is a command for every page, but there’s no way to navigate “relatively” such as going “back” or “forth”; in other words, there’s no `cd` command.

```
[REDACTED]
```

Pipelining

Introduced by Douglas McIlroy, pipelining enables commands to compose with each other parsibly simply via textual data. In this composition, there are relationships between commands to form the pipeline by parts. This composition is part of the design patterns in designing Unix programs as a whole [Ray03].

¹“flavorful” here means that the differences are poignant ones that the language writer feels when she transitions between the languages. See more at XXX

The *source* e.g. `feed`, `cat`, etc is the start of the pipeline, it exclusively produces output. The *sink* is the end of the pipeline, it exclusively consumes the input. In-between are *filters* e.g. `grep`, `rg`, `gron`, `sort`, `split`, `cut`, etc where data are passed in-and-out and get transformed in the process². So, a simple pipeline is one with a structure source-filter-sink. A complex pipeline is one that can “branch” ; the way to branch is to organize each pipeline branch as each group of commands as in `@fn`, then use special filters to branch on conditions using redirections with named pipes³. The rest are *cantrips* e.g. `set`, `man`, etc—commands that don’t produce output but instead apply an effect onto the current environment.

```
[REDACTED]
```

Function. For a special case, a reusable pipeline or grouping of commands would be called a *function*. In the bkhack shell language, functions aren’t allowed to be defined conventionally. Still, function is a useful abstraction; for example, since `feed` can be treated as a function simply built-in, it’s possible to customize the behavior of `feed` by overloading it. Indeed, the default `feed` command in bkhack, which automatically has limiting of 15 items pagination, is simply a function

```
feed() { feed | split -c 15 | cut -f1 |  
sort --hot ;}
```

²in the context of a pipeline-based system architecture, these are called *middle-wares*

³in practice, this is rarely done

⁴This is contrary to the semantics of pipeline seen elsewhere e.g. functional programming.

<program>	::= <command> (<program>)?	<i>Simple program</i>
	{ <program> <end of seq> } (<program>)?	<i>Composite program</i>
	(<program>) (<program>)?	<i>Subshell</i>
<command>	::= <word> (<command>)?	
<end of seq>	::= ;	
<word>	::= Omitted	
<seq connective>	::= ;	<i>Default continuation</i>
	&&	<i>Success continuation</i>

Table 1: Grammar for the bkhack shell language

which the user can customize. The rest of bkhack shell commands are exposed like so, thus the settings system.

Syntax

Sometimes, it is needed to parse the shell language from a text. The grammar of the bkhack shell language, given in EBNF form in Table 1. Notice how the command rule do not distinguish parts into flags or values which are expected from most real-world usage. Indeed, the responsibility of interpreting these parts, a.k.a. *options*, is left to the higher-level *option parser*, such as POSIX getopt. Parsing is implemented based on the grammar at Table 1.

In the initial design, we have only one phase for our parser. This is an eager, AST-less, final a.k.a. syntax-directed parser—effects are eagerly evaluated as each term is parsed. Eventually, we did make it so that it outputs an AST, for reusability and convenience.

In the next design, we have two phases for our parser: the lexical analysis phase (bộ phân tích từ vựng [Pha])⁵ and the parsing phase. The lexical analysis phase is to guarantee a minimal schema for the parsers so that they won't diverge from the language spec. This is an *implicit structure*. In contrast, we could have defined an ADT or GADT to make an explicit structure. However, our parsers won't be final, we would make a compromise on performance. Not only that, an implicit structure allows room for undefined behaviors; each parser can do things in flexible ways that should hopefully achieve desirable results.

Indeed, the lexing phase was added when we branched the parser to support *pastelling*; so, two

parsers, a parser and a pastel. A shared lexing phase helps communicating that the two parsers share a structure, even if that structure is only infra-, and there's no explicit GADT or signature to explicitly enforce it.

The usefulness of this starts to show when we added a new feature: support for quoted words.



Real-time parsing

The bkhack shell pasteller achieves real-time syntax highlighting by being constructed purely out of React combinators that follows the grammar above (Table 1) and the implicit structure above.

Typing

Sometimes, it is useful to verify the shell language before evaluation. The typing of the bkhack shell language, given in Figure 1. This is useful when shell commands have to be embedded in a statically-typed programming language.

The combinators are designed based on the typing rules at Figure 1. An example usage of these combinators would be

```
module Feed (Syntax : Shell.Sym)
{ open Syntax
  let rec v = () =>
    { feed |@ Split_by.count(10) |@ nil ;}
    and v' = () => observe @@ v() }
```

where the |@ operator—accompanied by the nil word to mark EOL—is a semantic translation of |

⁵a.k.a. character grouping

$$\begin{array}{c}
\frac{}{\text{nil } () : \text{Program } T} \text{Zero} \\
\\
\frac{t : \text{Program } A}{\text{\$}\%(t) : \text{Cmd } A} \text{Subsection} \\
\\
\frac{x : \text{Cmd } T \quad x' : \text{Program } T}{x \parallel \% x' : \text{Program } T} \text{Cons} \\
\\
\frac{t : \text{Program } A}{\text{obs}(t) : A} \text{Observe}
\end{array}$$

Figure 1: Typing for the bkhack shell language

as part of an abstract algebra used to construct an embedded bkhack shell pipeline. We believe that this type system can be used for composition of commands. Indeed, *type-driven composition* allows us to implement and explore possibilities purely through types, where the type system serves as a real-time theorem checker that prompts our paths.

Completion

When writing a pipeline, it is nice to provide a language service where user can request their partially-written command to be replaced with corrected form. When press `[TAB]` user Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore doleamus, fieri.

Convention

why are commands named the way they are? this is the same problem in sh. the truth is that there will never be a scheme that everyone can agree on. most names are historical and highly contextual. unix is loved because people like its architecture and they try to adopt the names, not that the names are actually good. indeed, there's so much you can do to appeal to a demographic of people. communication is two-way between the user and the developer.

- [She] "Shell Command Language." IEEE, The Open Group. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html
- [AT&] "AT&T Archives: The UNIX Operating System." [Online]. Available: <https://youtu.be/tc4ROCJYbm0?t=296>

- [Ray03] E. S. Raymond, "Interfaces–Unix Interface Design Patterns," in *The Art of Unix Programming*, 2003, ch. 11, pp. 302–320.
- [Pha] T. Phan-Thị, "Phân tích từ vựng," in *Giáo trình Trình biên dịch*, Nhà xuất bản Đại học Quốc gia TP.HCM, ch. 2.6.